



Structure-augmented LLMs for High-Level Synthesis Pragma Optimization

Haocheng Xu
haochx5@uci.edu
University of California, Irvine
Irvine, California, USA

Ye Qiao
yeq6@uci.edu
University of California, Irvine
Irvine, California, USA

Phyo Pyae Moe Aung
ppaung@uci.edu
University of California, Irvine
Irvine, California, USA

Alok Mishra
alok.mishra@hpe.com
HPE Labs
Milpitas, California, USA

Pavana Prakash
prakash@hpe.com
HPE Labs
Milpitas, California, USA

Rolando Pablo Hong Enriquez
rhong@hpe.com
HPE Labs
Milpitas, California, USA

Adam Han Wu
adamhw@uci.edu
University of California, Irvine
Irvine, California, USA

Zhiheng Chen
zhiehnc5@uci.edu
University of California, Irvine
Irvine, California, USA

Dejan Milojicic
dejan.milojicic@hpe.com
HPE Labs
Milpitas, California, USA

Sitao Huang
sitaoh@uci.edu
University of California, Irvine
Irvine, California, USA

Abstract

Pragma insertion drives the quality of high-level synthesis (HLS) designs. Choosing the right directives demands expert knowledge and reasoning about loop nesting, data dependences, and memory layout. While existing large language models (LLMs) show promise in code generation, they lack explicit program-structure awareness, limiting their ability to suggest effective pragmas. We present PRISM, a novel structure-augmented LLM that closes this gap by adding compiler-grade structural reasoning to a pretrained, frozen code LLM. It combines three hierarchical program representations, Abstract Syntax Tree (AST), Control-Flow Graph (CFG), and Data-Flow Graph (DFG), injecting them into a specific transformer layer while keeping original code tokens in a separate stream. The cross-attention gate at the injection point allows falling back to the pretrained representation when its structural signal is unhelpful. On zero-shot evaluation in HLS-Eval, PRISM synthesizes $3.5\times$ as many kernels as Llama3-8B (26.9% vs. 7.7%), and on the kernels where it does succeed, it produces designs that are $2.31\times$ faster (geomean) than GPT-5-mini's. In the agentic flow, the PRISM codegen outperforms other baselines when optimizing complex code and drives the average normalized improvement across the HLS-Eval suite to 26.4%.

Keywords

LLM, HLS, Structure-aware, Pragma Optimization

ACM Reference Format:

Haocheng Xu, Ye Qiao, Phyo Pyae Moe Aung, Alok Mishra, Pavana Prakash, Rolando Pablo Hong Enriquez, Adam Han Wu, Zhiheng Chen, Dejan Milojicic, and Sitao Huang. 2026. Structure-augmented LLMs for High-Level Synthesis Pragma Optimization. In *2026 ACM/IEEE International Symposium on Machine Learning for CAD (MLCAD '26)*, September 07–09, 2026, Jeju Island, Republic of Korea. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3831599.3840318>

1 Introduction

High-Level Synthesis (HLS) raises the abstraction level of hardware design from register-transfer logic to C/C++, but the quality of the resulting hardware is almost entirely determined by a small set of optimization directives, pragmas. Whether a loop is pipelined, how aggressively it is unrolled, and how its arrays are partitioned across memory banks determine the throughput, latency, and resource footprint of the synthesized accelerator, often by an order of magnitude. A single misplaced `ARRAY_PARTITION` can leave a pipelined loop starved for memory bandwidth; a single `PIPELINE` on a loop with a carried dependence can inflate the initiation interval from one cycle to dozens. Currently, pragma insertion remains one of the most labor-intensive steps in the HLS design flow, and automating it is a central problem for machine learning in CAD [4, 27].

Automating this reasoning has been a central goal of machine learning for CAD. The dominant approach treats pragma selection as design-space exploration (DSE): enumerate candidate configurations, estimate the quality of results (QoR) of each with a fast surrogate, and search. AutoDSE [22] formalized the search; GNN-DSE [20], HARP [21], and IronMan [26] replaced the surrogate with a graph neural network that reads the program graph and predicts latency and resource usage with remarkable fidelity. These methods, although structure-aware, score or rank a fixed candidate



This work is licensed under a Creative Commons Attribution 4.0 International License.
MLCAD '26, Jeju Island, Republic of Korea
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2878-5/2026/09
<https://doi.org/10.1145/3831599.3840318>

configuration rather than generate new directives and require an external design-space-exploration loop.

Large language models (LLMs) excel at generic code generation, yet when they read a program, they see only a linear token sequence. What an LLM should see (as illustrated in Figure 1) for reliable HLS optimization is the complex, hierarchical structure of the code – its Abstract Syntax Tree (AST), Control Flow Graph (CFG), and Data Flow Graph (DFG) – which traditional compilers use to guarantee correctness and to reason about performance. Because this structural information is absent, LLMs frequently hallucinate illegal or sub-optimal *pragmas* that only fail during synthesis.

In this paper, we present a novel framework that injects *three* hierarchical program graphs – AST, CFG, and DFG – into a pre-trained code LLM. The graphs are encoded by the three parallel GNN streams, compressed into a compact token set, and merged with the LLM via a *gated cross-attention* layer that lets the model fall back to its generic coding knowledge when the structural signal is weak. By training the graph encoders and the LLM jointly but on separate parameter subsets, our method learns program structure and pragma intent as interacting yet independent modalities. Our contributions include:

- First joint integration of AST, CFG, and DFG of a code into an LLM for HLS pragma generation with a gated cross-attention mechanism that preserves the LLM’s baseline coding ability while exploiting structural information.
- Decoupled, jointly-trained architecture that treats program structure and optimization directives as separate but interacting modalities.
- It synthesizes $3.5\times$ as many kernels as its Llama-base ancestor (26.9% vs. 7.7%), and on the kernels where it does succeed, it produces designs that are $2.31\times$ faster (geomean) than GPT-5-mini’s.
- On the agentic flow, it outperforms other baselines when optimizing complex code and achieves the average normalized improvement across a 26-kernel suite to 26.4%.

The remainder of the paper is organized as follows: Section 2 discusses current state-of-the-art work, Section 3 describes our

structure-augmented LLM architecture, Section 4 details the training methodology and presents the experimental results, and Section 5 concludes with a discussion of future work.

2 Background

Existing research falls into two complementary strands: (a) learning-based surrogate models that predict Quality-of-Result (QoR) for a given pragma configuration, and (b) large language models (LLMs) that edit or generate source code but typically ignore the underlying program structure. We briefly review each strand and highlight the gap our work fills.

2.1 Learning-Based HLS Design-Space Exploration

To avoid invoking the synthesizer for every candidate, many works train surrogate models that predict QoR from a program representation. AutoDSE [22] framed the problem as a guided search over the pragma space. GNN-DSE [20] builds a graph from the kernel’s LLVM IR and trains a GNN to regress latency and resource usage; a downstream search loop then queries the surrogate instead of the synthesizer. HARP [21] refines this idea with a hierarchical graph that isolates program semantics from pragma nodes, thereby improving transfer across kernels. IronMan [26] coupled a GNN predictor with reinforcement learning to drive the search itself. Although accurate and structurally faithful, these models are fundamentally discriminative: they evaluate only configurations proposed by an external search procedure and never emit an optimized source.

Our structure-augmented model complements these surrogates by directly producing a pragma-annotated kernel; a surrogate such as HARP could subsequently be used as a verifier or reward signal.

2.2 LLMs for Code and for Hardware Design

Instruction-tuned code LLMs have made code editing and generation accessible via natural language. Code Llama [19] and StarCoder [7] are strong open baselines, and instruction-tuning corpora such as InstructCoder [13] adapt general-purpose LLMs to edit existing code rather than generate it from scratch. In the hardware

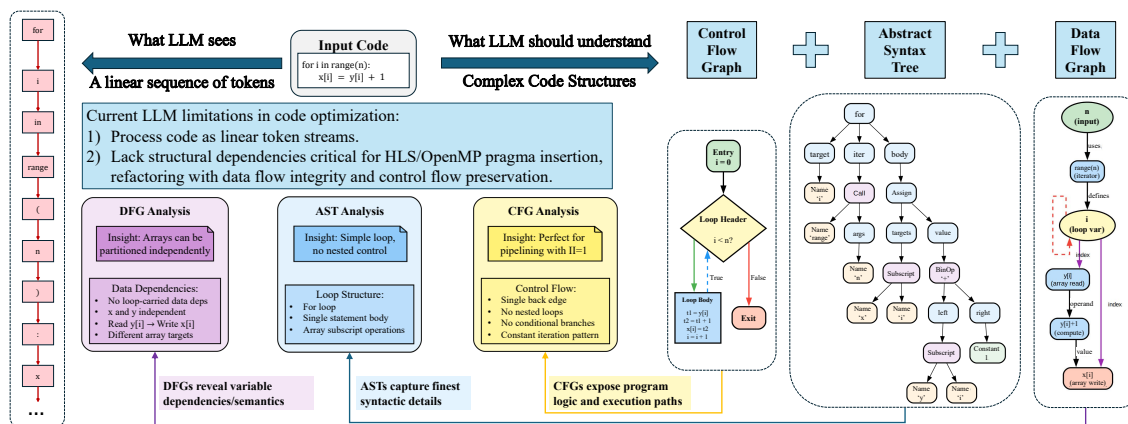


Figure 1: What LLM Fails to See in the Code.

domain, ChipNeMo [6] adapts LLMs to EDA tasks through domain pretraining, and several systems apply LLMs to RTL and HLS generation [8, 14, 23]. All of these systems consume code as a flat token sequence. While sufficient for surface-level edits, this representation provides no explicit view of control- and data-flow — information that is essential for deciding whether a pragma is legal and beneficial.

2.3 Structure-Aware Models for Code

The intuition that programs are graphs, not just strings, predates LLMs. Code2vec [2] learned representations from AST paths; GraphCodeBERT [10] added data-flow edges to a transformer encoder during pretraining; and StructCoder [24] conditioned a sequence-to-sequence model on AST and DFG signals. These works establish that structure helps, but they share three properties that limit their use for HLS. First, they inject structure on the encoder side, at pretraining time, which couples the structural component to a particular base model and an expensive training run. Second, they use a single graph view (usually the DFG, whereas a compiler reasons over syntax, control, and data flow together). Third, they target understanding and short edits, not the long, fragile directive blocks HLS requires.

3 Structure-augmented LLMs

3.1 Overview

As shown in Figure 2, PRISM (Pragma Reasoning via Integrated Structural Models) augments a pretrained, frozen code LLM with an explicit structural view of its input. Given a source kernel s and an optimization instruction, our structure-augmented model proceeds in four stages. First, it extracts three complementary graphs (AST, CFG, DFG) from s . Second, each graph is processed by its own GATv2 encoder [3], yielding a fixed-length embedding $z_g \in \mathbb{R}^{256}$ (Section 3.3). Third, the three embeddings are concatenated and projected into a bank of $k = 128$ graph tokens $T \in \mathbb{R}^{k \times d}$ that reside in the LLM’s embedding space (Section 3.4). Finally, the LLM’s text hidden states attend to T through a gated cross-attention module; the resulting enriched states are fed to the remaining transformer layers for decoding (Section 3.5). The complete data path is shown in Figure 2. We instantiate the base model as Llama-3.1-8B-Instruct with hidden size $d=4096$; all of its weights are frozen, and only the graph encoders, the projector, the cross-attention and gating parameters, and low-rank adapters (LoRA) are trained.

Two principles shape the design, and each anticipates a failure mode we wanted to avoid: **Decouple structure from directives**. The text stream carries the instruction and the kernel as

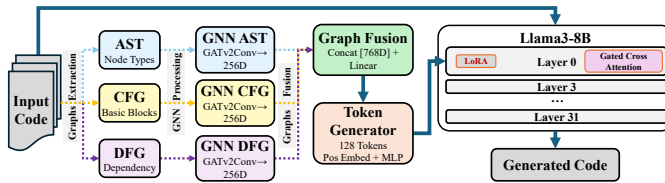


Figure 2: Structure-augmented LLM with Layer-specific Graph Injection

the model learned to read them; the graph stream carries an orthogonal, compiler-grade view of the same kernel, keeping the two separate until they are fused, rather than serializing the graph into the prompt. Let each be learned with its own parameters, and let the model learn their interaction explicitly, rather than forcing the structure through the same tokenizer that already struggles with it. **Preserve the pretrained model.** A code LLM’s value lies in weights that took enormous compute to train. Any structural addition that overwrites them trades one competence for another. PRISM therefore freezes the base model and admits structural signal only through gated residual paths (Section 3.5), so that in the limit of an unhelpful gate, the model reduces exactly to the original LLM.

3.2 Graph Construction

From the source kernel, we extract three different graphs, each exposing a distinct facet of the program, illustrated for a simple loop in Figure 1. The abstract syntax tree (AST) captures the hierarchical syntax, such as loop nests, subscript expressions, and operator structure. It shows that for the loop $x[i] = y[i] + 1$, the body is a single statement over array subscripts. The control-flow graph (CFG) records execution paths, including basic blocks, back edges, and branch conditions, making loop trip counts and nesting readily visible. Finally, the data-flow graph (DFG) represents definition-use relationships, thereby exposing loop-carried dependencies and array read/write patterns. In this case, it exposes that x and y are independent with no loop-carried dependence, which is precisely the fact that licenses partitioning the arrays and pipelining at $II=1$. These are the inferences an HLS expert makes at a glance, and that a token-streaming model has no direct way to make.

For Python kernels, the AST is obtained from the native `ast` module, and the CFG and DFG are constructed by a single traversal of that tree. The C/C++ extraction pipeline is described in Section 3.7. Each node receives a feature vector formed by concatenating a one-hot encoding of its node type, a hash of its surface token, and its degree. This yields a per-graph node-feature matrix $X_g \in \mathbb{R}^{n_g \times f}$ and an edge-index E_g for $g \in \{\text{AST}, \text{CFG}, \text{DFG}\}$.

3.3 Per-Graph GNN Encoders

Each graph is encoded independently by a stack of GATv2 [3] convolutions. GATv2 is chosen over the original GAT because its dynamic attention can express edge-conditioned importance, which matters when, for example, a single back edge in the CFG must dominate the loop’s representation. A stack uses multi-head attention (four heads) in the first layer, projects to a single head in the second, applies graph normalization, and pools the node embeddings into a graph-level vector $z_g \in \mathbb{R}^{256}$:

$$z_g = \text{Pool}(\text{GraphNorm}(\text{GATv2}(X_g, E_g))), \quad g \in \{\text{AST}, \text{CFG}, \text{DFG}\}. \quad (1)$$

Encoding the three graphs in parallel with separate parameters prevents cross-modal contamination and lets the subsequent fusion module learn explicit interactions among the modalities.

3.4 Graph Fusion and Token Generation

The three graph vectors are concatenated into a single structural descriptor $z = [z_{\text{AST}}; z_{\text{CFG}}; z_{\text{DFG}}] \in \mathbb{R}^{768}$, which a learned *graph token generator* expands into a bank of $k = 128$ tokens in the LLM

embedding space. Concretely, a linear projection followed by a positional embedding and an MLP maps z to $G \in \mathbb{R}^{k \times d}$:

$$G = \text{MLP}(\text{Proj}(z) + P), \quad G \in \mathbb{R}^{128 \times 4096}, \quad (2)$$

where P are learned positional embeddings over the k token slots. Expanding a single 768-d descriptor into k tokens (rather than one) gives the cross-attention layer multiple structural “slots” to attend to, which we found necessary for the model to route different aspects of structure to different parts of the generated kernel.

3.5 Gated Cross-Attention Injection

We inject the graph tokens at layer 0. By front-loading all structural information, we guarantee that every subsequent layer receives an enriched representation, while keeping the pre-trained weights intact through a lightweight gating mechanism that controls how much of the new token the model can use.

Let $H \in \mathbb{R}^{m \times d}$ be the text hidden states over the m input tokens and $G \in \mathbb{R}^{k \times d}$ the graph tokens. The module first computes a standard cross-attention with the text states as queries and the graph tokens as keys and values,

$$A = \text{softmax}\left(\frac{(HW_Q)(GW_K)^T}{\sqrt{d_k}}\right)GW_V, \quad (3)$$

and then blends the attended structural features A with the original states H under a learned, element-wise gate:

$$\gamma = \sigma(\text{MLP}([H; A])), \quad (4)$$

$$H' = \gamma \odot A + (1 - \gamma) \odot H. \quad (5)$$

Here σ is the logistic function and $\odot$ is the Hadamard product, so $\gamma \in (0, 1)$ controls, per position and per channel, how much structural signal is admitted. When $\gamma \rightarrow 0$ the layer reduces to the identity and the model behaves exactly as the pretrained LLM; this is what allows our structure-augmented model to add structure without overwriting the base model’s coding ability and what keeps training stable, since the network can ignore an unhelpful graph signal rather than being forced to use it. The enriched states H' are then passed to the remaining transformer layers and decoded.

3.6 Model Fine-tuning

PRISM is trained end-to-end with the base LLM frozen. We attach LoRA [12] adapters to the transformer layers and update only the adapters, the three GNN encoders, the token generator, and the cross-attention and gating parameters. Freezing the base both slashes the trainable parameter count and guards against catastrophic forgetting of the pretrained coding knowledge that PRISM is built to preserve. Because the components converge at different rates, for example, a randomly initialized GNN learns faster and noisier than a LoRA adapter on a converged backbone. Thus, we give each component its own learning-rate schedule, pacing the graph encoders and cross-attention heads to track the LoRA updates so that the structural and textual pathways advance together rather than one destabilizing the other. We fine-tune on InstructCoder [13], a corpus of over 114k instruction–input–output Python triples for code editing, optimization, and refactoring, which matches PRISM’s edit-an-existing-kernel usage. There is no additional fine-tuning on the C/C++ HLS pragma corpus.

3.7 Cross-Language Structural Transfer

Because the graph encoders were trained on Python, applying our structure-augmented model to C/C++ HLS kernels requires bridging two graph “languages.” We first strip macros, pragmas, and comments from the C source, rebuild the AST, CFG, and DFG over the cleaned code, and map C node types onto the Python node-type vocabulary so the encoders receive embeddings drawn from their training distribution. The mapping is imperfect because C exposes constructs with no Python analogue, notably explicit variable declarations and initializations, and these mismatches account for residual error quantified in Section 4. In all experiments, we retain the bare (pragma-free) kernel as the structural source, so the graph stream encodes the program to be optimized rather than any particular pragma choice, aligning with our model’s goal of conditioning generation on input structure.

4 Experiments and Results

Our evaluation answers two questions. First, does the structural signal enable the model to generate a synthesizable, optimized pragma block in a single shot where the baseline model fails (Section 4.1)? Second, does this advantage persist (or change character) when the model iterates and evolves within an agentic flow (Section 4.2)?

4.1 Zero-shot evaluation

4.1.1 Setup. We compare PRISM against three baselines that span open and closed, small and large: **Llama-3.1-8B** [9] (the base of PRISM); **gpt-oss-20b** [16], a 20B-parameter open model served locally; and **GPT-5-mini** [15], a frontier proprietary model served via the OpenAI API. Local models are evaluated on a single NVIDIA 4090 GPU. Each model is asked to emit a pragma block for a kernel; the generated source then runs through the HLS toolchain.

We use the HLS-Eval suite [1] of 106 kernels drawn from five benchmark families: PolyBench [17], MachSuite [18], CHStone [11], Rosetta [28], and ForgeBench [25]. The four models attempt slightly different subsets (some kernels produce malformed generations under one model and not another), so for an apples-to-apples comparison, we restrict the pass-rate and latency analyses to the 78-kernel intersection that every model attempted. We score three outcomes per generation: **Compile** — the emitted C compiles and passes a software test bench; **Synth** — the design synthesizes to RTL under the HLS tool; and report **Latency** for synthesized designs.

4.1.2 Python Benchmark. Before turning to HLS, we verify that the structural signal does not degrade general code-editing ability. We evaluate PRISM and the Llama-8B [9] baseline on HumanEval [5], 164 hand-written Python tasks with reference tests. We measure syntactic validity (via `ast.parse`) and Pass@1 under two output-handling strategies: *simple concatenation*, which appends the raw generation, and *clean extraction*, which truncates at natural stopping markers. The distinction matters: it isolates whether gains come from reasoning or from generation discipline.

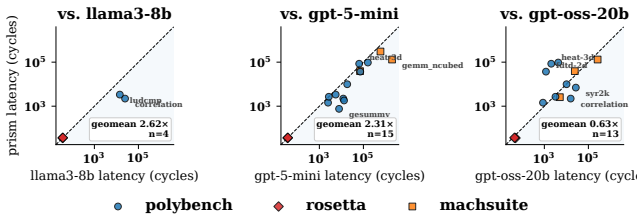
As shown in Table 1, under clean extraction, the two models are statistically tied (97.6% vs. 98.8% Pass@1). Under simple concatenation, the Llama baseline collapses to 55.5% Pass@1 because 93.9% of its outputs over-generate spurious test code; PRISM holds

Table 1: HumanEval Llama3-8B vs PRISM

Model	Syntax Validity		PASS@1		Avg. Time (s)
	Simple	Clean	Simple	Clean	
Llama3-8B	63.4%	97.6%	55.5%	97.6%	6.75
PRISM	98.2%	98.8%	97.0%	98.8%	20.45

at 97.0%, with over-generation in only 1.8% of outputs. The 41.5-percentage-point gap (about a 75% relative improvement) is therefore a generation-discipline effect (structural grounding teaches the model where the program ends) rather than a reasoning lift on easy Python. The inference-time cost is real but bounded: PRISM runs about 3× slower per query than the base (20.45 s vs. 6.75 s in our setup).

4.1.3 HLS Synthesis Pass Rates. Table 2 reports Synth and Compile rates over the 78-kernel intersection across all four models. We found **1) Structural augmentation lifts a small base model substantially.** Going from Llama-3.1-8B (7.7% synth) to PRISM (26.9% synth) is a 3.5× improvement on the kernels both models attempted with structural augmentation (Section 3). The gain is consistent across PolyBench [17], MachSuite [18], and Rosetta [28]. **2) Frontier and larger open baselines retain a coverage advantage.** GPT-5-mini reaches 52.6% and gpt-oss-20b reaches 43.6% on the same intersection — both higher than PRISM’s 26.9%. We do not claim PRISM beats them on raw synth coverage; the contribution is elsewhere (Section 4.1.4). What this comparison does establish is that an 8B parameter, locally-served, structurally-grounded model closes a meaningful fraction of the gap to a far larger proprietary model without relying on its scale (roughly 2.5× fewer parameters). **3) Per-family behavior isolates where structure pays.** PRISM outperforms gpt-oss-20b on MachSuite (43.8% vs. 31.2%), where loop and array patterns are rich enough for the structural signal to exploit. It trails on PolyBench, whose regular nested loops are forgiving enough that scale alone suffices, and collapses to 0% on CHStone, which is a benchmark heavy in C-specific constructs (struct definitions, typedefs, explicit pointer arithmetic) for which the Python-trained graph encoders have no analogue. This is exactly the failure mode predicted by our cross-language transfer scheme (Section 3.7). The fact that GPT-5-mini reaches 11.1% on CHStone isolates the residual gap to precisely the construct class the node-type mapping does not cover, and points to native C/C++ graph encoders as the clearest path to closing it (Section 5). ForgeBench remains near zero for all 4 models and yields only one synthesizable design, even for GPT-5-mini, which needs further investigations.

**Figure 3: Latency Speedup of PRISM**

4.1.4 HLS Latency Quality. Synth rate alone does not characterize the quality of a generated design. A pragma block that synthesizes may produce RTL that meets the toolchain’s correctness gate but barely improves on the no-pragma baseline. To compare quality directly, Figure 3 plots, for each baseline, PRISM’s synthesis cycle count against the baseline’s on the kernels where both synthesized. Both axes are log-scale; the dashed line is $y = x$; the shaded region below the line is where PRISM produces a faster design. **Against the Llama-base ancestor, PRISM is uniformly faster** (geomean 2.62×, $n=4$). Every shared synthesized kernel sits below the diagonal. Structural augmentation does not merely raise the chance of a legal design; it raises the quality of the legal designs it produces. **Against GPT-5-mini, PRISM is faster on the geomean despite the frontier model’s coverage advantage** (geomean 2.31×, median 1.80×, $n=15$). The largest single-kernel wins are machsuite__gemm_ncubed (15.2×), polybench__gesummv (10.3×), polybench__covariance (7.3×), and polybench__correlation (5.5×). The single notable loss is polybench__heat-3d (0.76×). The pattern is consistent: on kernels with the loop-and-array structure the graph stream can exploit, PRISM’s smaller, structurally-grounded model finds optimizations that a roughly order-of-magnitude larger general model misses. **Against gpt-oss-20b, results are mixed** (geomean 0.63×, median 1.19×, $n=13$). PRISM wins on the median kernel but loses heavily on three PolyBench stencils (heat-3d (0.02×), jacobi-2d (0.04×), and fdt-d-2d (0.03×)) for which gpt-oss-20b reports cycle counts an order of magnitude below what the iteration counts alone would predict. These designs pass synthesis and the test bench, but the reported latencies suggest an aggressive scheduling that may compress the iteration space rather than execute it; we report the geomean as-is for transparency and flag this caveat for future investigation.

4.2 Iterative Refinement via Agentic Adaptation

The zero-shot results establish that PRISM produces high-quality designs when it succeeds but at lower coverage than larger baselines (Section 4.1). In practice, HLS optimization is iterative: a designer (or an agent) proposes a pragma plan, reads the synthesizer’s feedback, and revises. We now ask how structural conditioning behaves inside that loop. We report two complementary experiments. Section 4.2.2 runs each model as both planner and codegen in a closed loop on three kernels of increasing structural complexity, exposing per-kernel convergence dynamics. Section 4.2.3 factorizes the loop into a 5×26 planner-codegen study that decouples the two roles across 26 kernels and isolates where the structural signal pays.

4.2.1 Setup. The agentic loop alternates between a *planner* role that proposes a pragma strategy in natural language and a *codegen* role that emits the pragma block. After each iteration, the design is synthesized; the planner receives the synthesis log and achieved latency in the next round. We run ten refinement iterations per kernel in Section 4.2.2 and three iterations in Section 4.2.3. We report two metrics: **improvement%** = $100 \times (1 - \text{best latency} / \text{baseline latency})$, normalized per kernel; and **synth pass rate**, the fraction of kernels for which the loop produces at least one synthesizable design. The five configurations in Table 3 factorize structural conditioning across the planner and codegen,

Table 2: Zero-shot pragma-insertion pass rates on the intersection of 78 kernels that every model attempted (apples-to-apples). Synth: the generated kernel synthesizes to RTL under the HLS tool. Compile (in italics, denoted *cmp*): the generated C compiles and passes the software test bench.

Model	Polybench(24)	Machsuite(16)	Chstone(18)	Rosetta(8)	Forgebench(12)	Overall(78)
PRISM	37.5% (9/24) (92% <i>cmp</i>)	43.8% (7/16) (62% <i>cmp</i>)	0.0% (0/18) (33% <i>cmp</i>)	62.5% (5/8) (62% <i>cmp</i>)	0.0% (0/12) (0% <i>cmp</i>)	26.9% (21/78) (55% <i>cmp</i>)
Llama3-8B	12.5% (3/24) (75% <i>cmp</i>)	6.2% (1/16) (44% <i>cmp</i>)	0.0% (0/18) (0% <i>cmp</i>)	25.0% (2/8) (25% <i>cmp</i>)	0.0% (0/12) (0% <i>cmp</i>)	7.7% (6/78) (35% <i>cmp</i>)
gpt-oss-20B	79.2% (19/24) (83% <i>cmp</i>)	31.2% (5/16) (75% <i>cmp</i>)	16.7% (3/18) (50% <i>cmp</i>)	87.5% (7/8) (88% <i>cmp</i>)	0.0% (0/12) (42% <i>cmp</i>)	43.6% (34/78) (68% <i>cmp</i>)
gpt-5-mini	91.7% (22/24) (100% <i>cmp</i>)	62.5% (10/16) (88% <i>cmp</i>)	11.1% (2/18) (56% <i>cmp</i>)	75.0% (6/8) (88% <i>cmp</i>)	8.3% (1/12) (17% <i>cmp</i>)	52.6% (41/78) (73% <i>cmp</i>)

Table 3: Agentic adaptation over 26 kernels with 3 iterations. Synth/Correct are the fractions of runs that synthesize / pass functional verification. Avg. Improvement is the average of normalized improvement over all kernels. Total Tokens is total tokens consumed. Correct/M Tok is the amount of correct design per million tokens.

Config	Planner	Codegen	Synth	Correct	Avg. Improvement	Total Tokens	Correct/M Tok
A	Llama-base	PRISM	11.7 %	11.1 %	9.5%	622,353	5.36
B	LORA-TEXT	LORA-TEXT	13.0 %	11.3 %	14.2%	407,443	8.32
C	LORA-TEXT	PRISM	19.9 %	19.9 %	14.1%	631,691	9.45
D	GPT-5-mini	PRISM	27.8 %	27.8 %	26.4%	1,442,718	5.77
E	PRISM	PRISM	32.2 %	32.2 %	0%	437,377	22.1

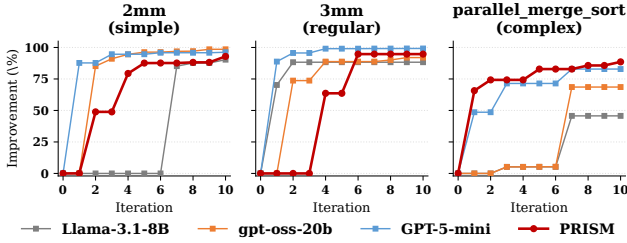


Figure 4: Agentic curve - Improvement vs iterations

with LORA-TEXT denoting the same checkpoint as PRISM but served with the graph encoder disabled.

4.2.2 Convergence depends on structural complexity. We first run single-model loops in which each model serves as codegen with gpt-5-mini as planner on three kernels of increasing structural complexity: 2mm (regular triply-nested matrix chain), 3mm (slightly larger but similarly regular), and parallel_merge_sort (irregular control flow with recursive splitting and dynamic memory access patterns). Figure 4 plots improvement over the baseline against iteration count.

The model ordering reshuffles with kernel complexity. On 2mm, GPT-5-mini and gpt-oss-20b reach 50% improvement within one to two iterations while PRISM lags to iteration 4 and finishes mid-pack (92.9% vs. 98.6% for the leader). On 3mm the ordering is similar. On parallel_merge_sort, the picture inverts. PRISM is the only model to improve on iteration 1 (reaching 65.7% on its first pragma proposal) reaches the 50% threshold three iterations earlier than

GPT-5-mini and six earlier than gpt-oss-20b, and finishes at the highest final improvement (88.6% vs. 82.9% for GPT-5-mini, 68.6% for gpt-oss-20b, and 45.7% for the Llama-3.1-8B baseline). The structural priors deliver their advantage exactly where the underlying task most rewards structural reasoning, mirroring the per-family pattern observed in zero-shot synthesis.

We are careful not to overclaim: PRISM is not the strongest single-model agentic loop in absolute terms. It is the strongest when the kernel genuinely benefits from structural reasoning, and its iteration efficiency scales with that structural payoff. This dynamic motivates the factorized study below, which asks whether structural conditioning is best deployed as a planner or as a codegen step.

4.2.3 Where structural conditioning belongs. Table 3 aggregates over all 26 kernels, with three findings that stand out.

Finding 1: structural codegen lifts legality without distorting the planner’s strategies, and it scales with planner strength. Configurations B and C share the LORA-TEXT planner and differ only in the codegen. Swapping the codegen from LORA-TEXT to PRISM raises the correct rate from 11.3% to 19.9% (a 8.6 percentage – point gain) and correct per million tokens from 8.32 to 9.45 while leaving the average improvement essentially unchanged (1% difference). The structural signal makes the codegen better at producing legal, synthesizable pragmas while maintaining similar optimizations the planner asks for. From A to C to D, correctness and improvement steadily increase when the planner gets stronger, which suggests PRISM’s scalability.

Finding 2: structural planning sacrifices optimization for legality. Configurations D and E share the PRISM codegen and

differ only in the planner. Although the PRISM planner (E) generates a larger share of correct designs than the GPT-5-mini planner (D) (32.2% vs. 27.8%), it shows no average improvement, whereas D achieves a 26.4% improvement. This is expected: PRISM has been fine-tuned as a structure-conditioned model, so its performance degrades in the purely text-based planning stage. The structurally grounded planner converges quickly to a single safe plan that satisfies the synthesizer but fails to push the QoR on any kernel beyond what the baseline already achieves. Plan diversity, which natural-language reasoning supplies and structural conditioning suppresses, is what enables deep optimization in the iterative loop. We also found GPT-5-mini as the best Planner in all cross-validation runs.

Finding 3: structural conditioning is a role-specific tool.

Read together, the $B \rightarrow C$ and $D \rightarrow E$ ablations identify where the structural signal belongs. The codegen role rewards legality, and structural conditioning supplies that legality without cost. The planner role rewards strategy diversity, and structural conditioning withdraws it. The best configuration in the suite is therefore D: a natural-language planner paired with the PRISM codegen. We did not anticipate this result at the start of the project, where the symmetric design (PRISM on both sides) was the natural default, and we believe the asymmetric lesson it teaches is the most transferable contribution of our agentic study. Any system that adds a structural adapter to an agentic LLM loop should ask, for each role, whether that role rewards exploration or rewards constraint, and place the adapter accordingly.

An efficiency note. Token totals in Table 3 do not track quality of result monotonically. The most successful configuration (D) is also the most expensive (1.44 M tokens), and the cheapest PRISM-codegen configuration (E) produces no improvement at all. Tokens are the cost of exploration in this setting, and exploration is what differentiates D from E. A practitioner choosing between D and the more economical C trades roughly 2.3× the token budget for roughly 1.9× the average improvement and 1.4× the correct rate – a steep but justifiable bill for kernels where QoR matters.

5 Conclusion

We presented PRISM, a framework that augments a pretrained, frozen code-LLM with explicit structural reasoning for HLS-pragma optimization. By encoding the kernel’s AST, CFG, and DFG with three parallel GNN streams and injecting them at a specific transformer layer through gated cross-attention, PRISM bridges the long-standing divide between structure-aware models that cannot generate and generative models that are structure-blind. In zero-shot evaluation, it synthesizes 3.5× as many kernels as its Llama-base ancestor (26.9% vs. 7.7%), and on the kernels where it does succeed, it produces designs that are 2.31× faster (geomean) than GPT-5-mini’s. When paired with a planner in agentic flow, the PRISM codegen outperforms other baselines when optimizing complex code and drives the average normalized improvement across 26 kernels in HLS-Eval suite to 26.4%.

Acknowledgment: we would like to thank the support from Hewlett Packard Enterprise, NSF (Awards No. 2443992), and AMD.

References

- [1] Stefan Abi-Karam and Cong Hao. 2025. HLS-Eval: A Benchmark and Framework for Evaluating LLMs on High-Level Synthesis Design Tasks. In *2025*

- IEEE International Conference on LLM-Aided Design (ICLAD)*. IEEE, 219–226. doi:10.1109/iclad65226.2025.00021
- [2] Uri Alon, Meital Zilberstein, Omer Levy, and Eran Yahav. 2018. code2vec: Learning Distributed Representations of Code. arXiv:1803.09473 [cs.LG] <https://arxiv.org/abs/1803.09473>
- [3] Shaked Brody, Uri Alon, and Eran Yahav. 2022. How Attentive are Graph Attention Networks? arXiv:2105.14491 [cs.LG] <https://arxiv.org/abs/2105.14491>
- [4] Jason Cong, Bin Liu, Stephen Neuendorffer, Juanjo Noguera, Kees Visser, and Zhiru Zhang. 2011. High-Level Synthesis for FPGAs: From Prototyping to Deployment. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 30, 4 (2011), 473–491. doi:10.1109/TCAD.2011.2110592
- [5] Mark Chen et al. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG] <https://arxiv.org/abs/2107.03374>
- [6] Mingjie Liu et al. 2024. ChipNeMo: Domain-Adapted LLMs for Chip Design. arXiv:2311.00176 [cs.CL] <https://arxiv.org/abs/2311.00176>
- [7] Raymond Li et al. 2023. StarCoder: may the source be with you! arXiv:2305.06161 [cs.CL] <https://arxiv.org/abs/2305.06161>
- [8] Yonggan Fu, Yongan Zhang, Zhongzhi Yu, Sixu Li, Zhifan Ye, Chaojian Li, Cheng Wan, and Yingyan Celine Lin. 2023. GPT4AIGChip: Towards Next-Generation AI Accelerator Design Automation via Large Language Models. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. 1–9. doi:10.1109/ICCAD57390.2023.10323953
- [9] Aaron Grattafiori et al. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783* (2024). <https://doi.org/10.48550/arXiv.2407.21783>
- [10] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. 2021. GraphCodeBERT: Pre-training Code Representations with Data Flow. arXiv:2009.08366 [cs.SE] <https://arxiv.org/abs/2009.08366>
- [11] Yuko Hara, Hiroyuki Tomiyama, Shinya Honda, Hiroaki Takada, and Katsuya Ishii. 2008. CHStone: A benchmark program suite for practical C-based high-level synthesis. In *2008 IEEE International Symposium on Circuits and Systems (ISCAS)*. 1192–1195. doi:10.1109/ISCAS.2008.4541637
- [12] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685 [cs.CL] <https://arxiv.org/abs/2106.09685>
- [13] Kaixin Li, Qisheng Hu, Xu Zhao, Hui Chen, Yuxi Xie, Tiedong Liu, Qizhe Xie, and Junxian He. 2024. InstructCoder: Instruction Tuning Large Language Models for Code Editing. arXiv:2310.20329 [cs.CL] <https://arxiv.org/abs/2310.20329>
- [14] Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. 2023. RTLLM: An Open-Source Benchmark for Design RTL Generation with Large Language Model. arXiv:2308.05345 [cs.LG] <https://arxiv.org/abs/2308.05345>
- [15] openai. 2025. *GPT-5 mini*. <https://developers.openai.com/api/docs/models/gpt-5-mini>
- [16] openai. 2025. *GPT-OSS*. <https://huggingface.co/openai/gpt-oss-20b>
- [17] Louis-Noël Pouchet. [n. d.]. <https://www.cs.colostate.edu/~pouchet/software/polybench/>
- [18] Brandon Reagen, Robert Adolf, Yakun Sophia Shao, Gu-Yeon Wei, and David Brooks. 2014. MachSuite: Benchmarks for accelerator design and customized architectures. In *2014 IEEE International Symposium on Workload Characterization (IISWC)*. 110–119. doi:10.1109/IISWC.2014.6983050
- [19] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2024. Code Llama: Open Foundation Models for Code. arXiv:2308.12950 [cs.CL] <https://arxiv.org/abs/2308.12950>
- [20] Atefeh Sohrabizadeh, Yunsheng Bai, Yizhou Sun, and Jason Cong. 2022. Automated accelerator optimization aided by graph neural networks. In *Proceedings of the 59th ACM/IEEE Design Automation Conference (San Francisco, California) (DAC '22)*. Association for Computing Machinery, New York, NY, USA, 55–60. doi:10.1145/3489517.3530409
- [21] Atefeh Sohrabizadeh, Yunsheng Bai, Yizhou Sun, and Jason Cong. 2023. Robust GNN-Based Representation Learning for HLS. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. 1–9. doi:10.1109/ICCAD57390.2023.10323853
- [22] Atefeh Sohrabizadeh, Cody Hao Yu, Min Gao, and Jason Cong. 2021. AutoDSE: Enabling Software Programmers Design Efficient FPGA Accelerators. In *The 2021 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (Virtual Event, USA) (FPGA '21)*. Association for Computing Machinery, New York, NY, USA, 147. doi:10.1145/3431920.3439464
- [23] Shailja Thakur, Baleegh Ahmad, Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Ramesh Karri, and Siddharth Garg. 2023. VeriGen: A Large Language Model for Verilog Code Generation. arXiv:2308.00708 [cs.PL] <https://arxiv.org/abs/2308.00708>
- [24] Sindhu Tipirneni, Ming Zhu, and Chandan K. Reddy. 2024. StructCoder: Structure-Aware Transformer for Code Generation. *ACM Trans. Knowl. Discov. Data* 18, 3,

- Article 70 (Jan. 2024), 20 pages. doi:10.1145/3636430
- [25] Andy Wanna, Hanqiu Chen, and Cong Hao. 2025. ForgeBench: A Machine Learning Benchmark Suite and Auto-Generation Framework for Next-Generation HLS Tools. arXiv:2504.15185 [cs.AR]. <https://arxiv.org/abs/2504.15185>
- [26] Nan Wu, Yuan Xie, and Cong Hao. 2021. IronMan: GNN-assisted Design Space Exploration in High-Level Synthesis via Reinforcement Learning. In *Proceedings of the 2021 Great Lakes Symposium on VLSI (Virtual Event, USA) (GLSVLSI '21)*. Association for Computing Machinery, New York, NY, USA, 39–44. doi:10.1145/3453688.3461495
- [27] Haocheng Xu, Haotian Hu, and Sitao Huang. 2024. Optimizing High-Level Synthesis Designs with Retrieval-Augmented Large Language Models. In *2024 IEEE LLM Aided Design Workshop (LAD)*. 1–5. doi:10.1109/LAD62341.2024.10691855
- [28] Yuan Zhou, Udit Gupta, Steve Dai, Ritchie Zhao, Nitish Srivastava, Hanchen Jin, Joseph Featherston, Yi-Hsiang Lai, Gai Liu, Gustavo Angarita Velasquez, Wenping Wang, and Zhiru Zhang. 2018. Rosetta: A Realistic High-Level Synthesis Benchmark Suite for Software Programmable FPGAs. In *Proceedings of the 2018 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (Monterey, CALIFORNIA, USA) (FPGA '18)*. Association for Computing Machinery, New York, NY, USA, 269–278. doi:10.1145/3174243.3174255

A Artifact Appendix

A.1 Abstract

This artifact reproduces the two evaluation workflows reported in the paper: zero-shot HLS pragma generation (Section 4.1) and the iterative agentic flow built on Autocomp (Section 4.2). It provides both training and inference scripts for PRISM .

A.2 Artifact check-list (meta-information)

- **Algorithm:** Structure-augmented LLM decoding for HLS pragma insertion; AST/CFG/DFG graph encodings prepended as soft-prompt tokens to a LoRA-adapted Llama 3.1 8B.
- **Program:** Two CLI entry points (`artifact/run_zero_shot.py`, `artifact/run_agentic.py`), a HumanEval benchmark, and a cached-metric figure script.
- **Compilation:** AMD/Xilinx Vitis HLS 2023.1 (C simulation, test-bench, and synthesis) for the live evaluation paths only.
- **Model:** Llama 3.1 8B Instruct plus a graph-LoRA adapter; OpenAI gpt-5.4-mini-2026-03-17; gpt_oss_20b.
- **Model availability and access:** Base weights are gated under Meta's license. The adapter is archived at 10.5281/zenodo.21636764. API models are not archivable.
- **LLM prompts and inference settings:** Prompt construction code and per-run prompt dumps are included; inference settings are recorded in `artifact/configs/zero_shot_paper.json` and `agentic_paper.json`. See Section A.3.5.
- **Data set:** Instruct-Coder(training), HLS-Eval, ForgeBench. not redistributed.
- **Run-time environment:** Linux, Python 3.11, uv for the pinned CUDA environment. Docker image provided for the offline path.
- **Container, VM, or locked environment:** Dockerfile based on a digest-pinned python:3.11-slim; uv.lock pins the full inference environment.
- **Hardware:** NVIDIA GPU with ≥ 24 GB VRAM ; no GPU required for the offline path.
- **Run-time state:** Run directories carry resolved configs, job plans, incremental event logs, generated sources, and per-case evaluation records. Complete records are reused on restart.
- **Execution:** `docker run` for the offline path; `uv run` with environment variables for model and checkpoint paths for live reruns.
- **Metrics:** pass@1 over compile, testbench, and synthesis success; Vitis latency and LUT/FF/BRAM/DSP/URAM utilization.
- **Output:** `results.csv`, `summary.json`, `failures.json`, per-case evaluation.json, generated C++, and regenerated figures.

- **Expected results and tolerances:** See Section A.6. Exact LLM text reproduction is not expected and is not central to any claim.
- **Experiments:** Table 1 (HumanEval), Table 2 (zero-shot), Table 3 (agentic evidence scope), Figure 3 (metric comparison), Figure 4 (agentic iteration curves).
- **Proprietary EDA tools, PDKs, and licensed technology files:** AMD/Xilinx Vitis HLS 2023.1. Not included and not redistributable. No foundry PDK is required.
- **How much disk space required (approximately)?:** 30 GB for the offline path, with benchmarks, base model, and adapter staged.
- **How much time is needed to prepare workflow (approximately)?:** 10 minutes for the Docker offline path; 1 hour for the full environment including model downloads.
- **How much time is needed to complete experiments (approximately)?:** 2-3 days (full reruns).
- **API cost or GPU-hours required (if applicable)?:** \$20-50 depends on setting.
- **Publicly available?:** Yes.
- **Code licenses (if publicly available)?:** Apache-2.0.
- **Data/model licenses and usage restrictions (if applicable)?:** Adapter derives from Llama 3.1 and inherits Meta's license. ForgeBench is MIT. The pinned HLS-Eval revision declares no license, so its content is fetched rather than redistributed.
- **Workflow framework used?:** None; plain Python CLIs with JSON manifests.
- **Zenodo DOI for archived artifact:** 10.5281/zenodo.21636764

A.3 Description

Source repository: <https://github.com/HaochengX/GACO/tree/mlcad-2026-prism-artifact>. Archived record with the adapter bundle: 10.5281/zenodo.21636764. The Zenodo record is the version evaluated.

A.3.1 Hardware dependencies. The offline path requires only a Linux host capable of running Docker. Live reruns require a GPU with at least 24 GB of VRAM.

A.3.2 Software dependencies. Check README

A.3.3 Commercial software and PDK dependencies. AMD/Xilinx Vitis HLS 2023.1.

A.3.4 Data sets. Fetched and checked out at immutable revisions by `run_agentic.py` setup:

- HLS-Eval, <https://github.com/sharc-lab/hls-eval>
- ForgeBench, <https://github.com/hchen799/ForgeBench>
- Autocomp, <https://github.com/ucb-bar/autocomp>

A.3.5 Models. PRISM. Llama 3.1 8B Instruct (gated under Meta's license) with the graph-LoRA adapter `0910_layer1` (633 MB), archived at 10.5281/zenodo.21636764.

A.4 Installation

```
docker build -t prism-ae .
docker run --rm prism-ae
sudo docker run --rm -it prism-ae bash
python -m pytest tests/artifact -q
bash check_install.sh
```

The final command runs the offline test suite and is the smoke test. Expected output: e.g. "N passed, N skipped in Ns". The repository-hygiene tests skip inside the image because it contains no git check-out; this is expected. The image requires no network access at run

Table 4: Validation map for each key result.

Result	Command	Criterion	Path
Suite	<code>docker run prism-ae</code>	all pass/skip	offline
Fig. 3	<code>process_results.py</code>	plot	cached
Fig. 4	<code>run_agentic.py --preset figure4</code>	plot	cached
Table 1	<code>Humaneval_benchmark.py</code>	pass rate	full
Table 2	<code>run_zero_shot.py paper</code>	pass rate	full
Table 3	<code>--preset table3-evidence</code>	pass rate	full

time, which can be confirmed with `docker run --rm --network none prism-ae`.

A.5 Experiment workflow

`run_zero_shot.py` implements Section 4.1 as independent one-shot generation: one sample per (model, case), with no planner, search, feedback, repair, or baseline fallback. The full manifest holds 106 cases; Table 2 uses the 78-case intersection present in all four executed metrics files. Generation and evaluation are pipelined — a case’s generated source is consumed by a Vitis worker as soon as it is written atomically, so generation does not block on synthesis of the whole suite.

`run_agentic.py` implements the iterative Autocomp flow. The `figure4` preset covers 2mm, 3mm, and `parallel_merge_sort` over ten iterations. The `table3-evidence` preset covers the saved evidence scope: five configurations $\times$ 15 cases $\times$ two repetitions, at three Autocomp iterations per job, for 150 jobs.

A.6 Evaluation and expected results

From the committed metrics CSVs with no model, API, or synthesis involvement, `figure/figure3/process_results.py` regenerates Figure 3; `figure/figure4/plot_agent.py` regenerates Figure 4. `testing/Humaneval_benchmark.py` reproduce the benchmark results of table 1 for HumanEval.

Table 4 maps each paper result to the command that produces it, the expected output, the similar-result criterion, and the intended validation path. Paths are: *offline* (no credentials or licensed tools), *cached* (recomputed from committed records), and *full* (live models and synthesis).

A.7 Experiment customization

Individual lanes or cases can be selected by repeating `--model` or `--only`. Summaries can be recomputed from an existing run directory without re-invoking any model via `run_zero_shot.py summarize --run-dir <dir>`. Model and adapter locations are overridden through `GACO_MODEL_PATH` and `GACO_CHECKPOINT_PATH`. Setting `PRISM_AUTOCOMP_SOURCE` to a checkout of the pinned Autocomp commit enables offline verification that the provided patch still applies.

One can also use `training/train_fusion_single.py` to work on a customized adapter.

B Additional Details

B.1 Training Details

- **Hardware:** 2 $\times$ H100

- **Batch Size:** 14
- **Gradient accumulation:** 2 (Effective batch size 28)
- **Learning rate:** 1×10^{-6}
- **Optimizer:** AdamW, betas=(0.9,0.95), eps= 1×10^{-6}
- **Weight Decay:** 0.01
- **LR Scheduler:** Cosine Annealing
- **Precision:** bfloat16
- **LoRA:** rank=12, alpha=16, dropout=0.1
- **Target Modules:** q_proj, k_proj, v_proj, o_proj

B.2 Model Architecture

Component	Input Shape	Output Shape	Parameters
AST GNN	[x, 128]	[B, 256]	~200K
CFG GNN	[x, 128]	[B, 256]	~200K
DFG GNN	[x, 128]	[B, 256]	~200K
Fusion	[B, 768]	[B, 768]	~590K
TokenGen	[B, 768]	[B, 128, 768]	~1.2M
Graph→LLaMA	[B, 128, 768]	[B, 128, 4096]	~3.1M
LLaMA+LoRA	[B, L, 4096]	[B, L, vocab]	~100M (LoRA)

Overall trainable parameters ~106M

B.3 Limitations

First, the graph encoders add per-token inference overhead; the loop-level token efficiency offsets this in the agentic setting, but PRISM remains slower per forward pass than the vanilla base model. Second, the cross-language transfer from Python-trained graph encoders to C/C++ HLS sources relies on a node-type mapping that is necessarily imperfect (declarations and explicit initializations in C have no Python analogue), and this accounts for a measurable share of the residual failure rate. Third, PRISM requires a complete, parsable kernel: partial code or syntactically broken input cannot be reasoned about structurally, since the graph streams collapse before the gated cross-attention can act.

B.4 Future directions

The most natural extension is to drop the Python detour entirely and build graph encoders over Clang ASTs and LLVM IR for C/C++, thereby exposing pointer aliasing, memory hierarchy, and the explicit type information that the current node-type mapping discards. The same architecture should transfer to RTL design with Verilog and SystemVerilog by replacing the program graphs with module-hierarchy, signal-dependency, and timing-constraint graphs, and to SystemC by adding process-network and transaction-level representations. Two questions also remain open: (1) whether the flow we exploit (syntax, control, semantics) is universal across code-transformation tasks or specific to pragma optimization, and (2) whether a GNN surrogate such as HARP can serve as a downstream verifier or reward signal that closes the loop between PRISM’s generative front end and the QoR-aware discriminative models the HLS community has spent years building.